

BattingEdge

AI Cricket Batting Analysis System

Dataset Construction, Preprocessing & Feature Engineering

1. Dataset Construction

The development of the BattingEdge classification system required the assembly of a purpose-built video dataset, as no single publicly available repository provided adequate coverage of the five target shot classes in a format suitable for pose-based feature extraction. The five shot classes considered throughout all subsequent stages of the project are: Cover Drive, Cut Shot, Defense, Pull Shot, and Sweep Shot. Data acquisition was carried out across four distinct sources, each chosen to maximise class diversity, batsman variety, and environmental heterogeneity.

1.1 Data Sources

Source 1 — Cric10Shots Dataset: The primary source of labelled video data was the publicly available Cric10Shots repository, which contains annotated cricket batting footage spanning ten shot categories. From this dataset, only the five shot classes relevant to the BattingEdge scope were retained. Videos corresponding to the remaining five shot types were discarded entirely at this stage, ensuring that no out-of-scope label noise propagated into the downstream pipeline.

Source 2 — Hugging Face Repositories: Supplementary cricket video and image datasets hosted on the Hugging Face platform were incorporated to increase per-class sample counts and introduce additional variation in player physique, pitch background, and camera perspective. These datasets provided particularly useful coverage for shot classes that were underrepresented within the Cric10Shots corpus.

Source 3 — Self-Recorded Video Footage: To address coverage gaps for specific shot angles and batting stances that were absent from public repositories, a set of videos was recorded manually. These recordings featured multiple individuals executing each of the five target shots under controlled and semi-controlled conditions, thereby introducing real-world pose variation that improved model generalisation.

Source 4 — YouTube-Edited Clips: Cricket match highlights and coaching tutorial footage sourced from YouTube were trimmed and edited to isolate individual batting strokes. Only clips in

which a single batsman executed a clearly identifiable shot were retained. This source was particularly valuable for obtaining footage of professional-level stroke play, providing a high-quality reference distribution against which grassroots players could later be benchmarked.

2. Data Cleaning and Manual Sorting

Raw video material aggregated from the four sources described in Section 1 was subjected to a two-phase cleaning procedure before any automated processing was applied. This two-phase approach — first manual, then automated — was adopted to eliminate the most egregious quality failures rapidly through human judgement, and subsequently to enforce quantitative quality constraints programmatically.

2.1 Phase 1 — Manual Review and Class Folder Sorting

All videos were reviewed manually and assigned to one of five class-labelled folders on disk: `cover_drive/`, `cut_shot/`, `defense/`, `pull_shot/`, and `sweep_shot/`. During this review, clips were rejected if they exhibited any of the following disqualifying characteristics:

- Blurry or low-resolution footage in which body landmarks could not plausibly be resolved.
- Incomplete shot execution, such as a defensive block that was interrupted before the follow-through.
- Unfavourable camera angles — specifically, behind-the-wicket perspectives that produced near-lateral views of the batsman, rendering shoulder and hip orientation ambiguous.
- Multiple persons simultaneously visible within the principal playing area, which would cause the batsman-isolation step to select an incorrect subject.
- Heavy occlusion of the batsman by stumps, fielders, or overlaid broadcast graphics.

Only clips satisfying all of these criteria proceeded to Phase 2 automated cleaning.

2.2 Phase 2 — Automated Cleaning Script

Following manual sorting, a Python script was executed against each class folder to enforce quantitative quality constraints. The script performed the following operations on every video:

- **Pose-frame validation:** Minimum frame threshold enforcement

Each video was decoded frame-by-frame and passed through the YOLOv8n person detector and MediaPipe BlazePose pipeline. Any video that yielded fewer than 10 frames on which a valid full-body skeleton could be extracted was discarded, as sequences shorter than this threshold were deemed insufficient for reliable temporal modelling.

- **Minimum bounding-box size:** Crop-size validation

YOLO bounding boxes smaller than 64×64 pixels after padding were rejected on a per-frame basis, preventing the pose estimator from operating on crops too small to resolve joint landmarks accurately.

- **YOLO confidence threshold:** Detection confidence filtering

Only person detections with a YOLO confidence score of 0.4 or above were accepted, filtering out frames in which the batsman was partially occluded or at the periphery of the field of view.

3. Data Augmentation and Class Balancing

Following the two-phase cleaning procedure, the per-class sample distribution was found to be imbalanced, with certain shot types — particularly the Defense and Sweep Shot — being substantially underrepresented relative to the Cover Drive. An imbalanced training set would cause the classifier to develop a systematic bias toward majority classes, degrading recall for minority shot types. To address this, a targeted augmentation pipeline was applied exclusively to the video-level data prior to feature extraction, producing synthetic variants of underrepresented classes.

The following augmentation transformations were applied:

- Horizontal flipping to simulate left-handed batsman stance and mirror-image shot geometry.
- Brightness and contrast variation to replicate different lighting conditions across outdoor and indoor venues.
- Playback speed perturbation (slight slow-down and speed-up) to introduce temporal variation within each shot class.
- Gaussian noise injection to approximate low-quality recording conditions.

After augmentation, the final dataset comprised 3,935 samples distributed perfectly evenly across the five shot classes, with each class containing exactly 787 samples. This perfect class balance eliminates any bias the classifier might otherwise develop toward majority classes. The dataset was partitioned into three non-overlapping splits as summarised in Table 1 below.

Table 1: Dataset Split Summary

Split	Samples / Class	Total Samples	Percentage (%)	Purpose
Training Set	550	2,750	69.9%	Base model training
Validation Set	118	590	15.0%	Meta-Learner training (Stacking Ensemble)
Test Set	119	595	15.1%	Final held-out evaluation only
Total	787	3,935	100%	—

It is important to note that the Validation Set was utilised exclusively for training the Logistic Regression Meta-Learner of the Stacking Ensemble architecture. The Test Set was held strictly untouched throughout all training and tuning phases and was accessed only during the final evaluation pass. Crucially, the perfect per-class balance of 787 samples across all five shot classes (Cover Drive, Cut Shot, Defense, Pull Shot, Sweep Shot) means that accuracy metrics are directly comparable across classes without any correction for class imbalance.

4. Feature Engineering

Raw video data cannot be fed directly into a machine learning classifier. A structured feature engineering pipeline was designed and implemented to convert each video clip into a fixed-dimensional numerical representation that captured both the spatial geometry of the batsman's body posture and the temporal evolution of that geometry across the duration of the shot. This pipeline produced a 107-feature $\times$ 50-frame representation per video, resulting in a tensor of shape (50, 107) for each sample.

4.1 Batsman Isolation via YOLOv8

A critical preprocessing challenge in cricket video analysis is the presence of multiple persons within the frame — including fielders, the wicket-keeper, the umpire, and the non-striking batsman. Feeding the full frame directly into a pose estimator would produce ambiguous or incorrect skeletal detections. To resolve this, the YOLOv8 nano (YOLOv8n) object detector was applied to each frame prior to pose estimation.

YOLOv8n was configured to detect the person class exclusively (COCO class index 0) with a confidence threshold of 0.4. Among all detected persons, the bounding box with the largest pixel area was selected as the primary batsman — the rationale being that the striking batsman is

invariably the person closest to the camera and therefore occupies the greatest area within the frame. A padding of 20 pixels was added to each edge of the selected bounding box before cropping, ensuring that extremity landmarks such as wrists and ankles were not inadvertently clipped. Crops smaller than 64×64 pixels were discarded on a per-frame basis.

4.2 Pose Landmark Extraction via MediaPipe BlazePose

Following batsman isolation, each cropped frame was processed by MediaPipe BlazePose, configured with a model complexity of 1 (full-body mode) and minimum detection and tracking confidence thresholds of 0.5. BlazePose produces a set of 33 anatomically defined full-body landmarks, each described by three coordinates (x, y, z) representing normalised horizontal position, normalised vertical position, and relative depth respectively.

The 33-landmark skeleton provides comprehensive coverage of the human body relevant to cricket batting biomechanics, including bilateral shoulder, elbow, wrist, hip, knee, and ankle joints, as well as facial and torso reference points. Frames on which MediaPipe failed to detect any pose were silently discarded and did not contribute to the feature sequence.

4.3 Raw Coordinate Features (99 Values per Frame)

For each successfully processed frame, the (x, y, z) coordinates of all 33 landmarks were extracted and concatenated into a flat vector of 99 values. These raw coordinate features capture the absolute spatial configuration of the batsman's body within the cropped bounding box. Since the coordinates are expressed relative to the dimensions of the MediaPipe input image — which is itself the YOLO-cropped region centred on the batsman — the features are inherently approximately translation-invariant with respect to the batsman's position within the original video frame.

4.4 Biomechanical Angle Features (8 Values per Frame)

Raw landmark coordinates alone do not explicitly encode the angular relationships between body segments that are most diagnostically informative for batting technique assessment. To complement the coordinate features, eight biomechanical angles were computed per frame using geometric operations on the landmark set. These angles are described in Table 2 below.

Table 2: Biomechanical Angle Features Computed per Frame

Feature Name	MediaPipe Landmarks	Computation Method	Biomechanical Significance
Left Elbow Angle	11 → 13 → 15	$\arccos(\text{dot product of vectors } \mathbf{ba}, \mathbf{bc})$	Left arm extension during swing
Right Elbow Angle	12 → 14 → 16	$\arccos(\text{dot product of vectors } \mathbf{ba}, \mathbf{bc})$	Top-hand control and bat angle
Left Knee Angle	23 → 25 → 27	$\arccos(\text{dot product of vectors } \mathbf{ba}, \mathbf{bc})$	Front-leg drive and weight transfer
Right Knee Angle	24 → 26 → 28	$\arccos(\text{dot product of vectors } \mathbf{ba}, \mathbf{bc})$	Back-foot positioning and balance
Shoulder Tilt Angle	11 → 12 (bilateral)	$\arctan2(\Delta y, \Delta x)$ of shoulder vector	Shoulder alignment and rotation
Hip Tilt Angle	23 → 24 (bilateral)	$\arctan2(\Delta y, \Delta x)$ of hip vector	Hip rotation and body lean
Bat Swing Angle	11 → 15 (shoulder to wrist)	$\arctan2(\Delta y, \Delta x)$ of arm-swing vector	Bat trajectory and swing plane
Stance Width	27 & 28 (both ankles)	$ x_{\text{left_ankle}} - x_{\text{right_ankle}} $	Base width and foot separation

The four joint angles (elbows and knees) were computed using the standard three-point vector angle formula, as expressed in Equation 3.1. Given three landmark points a , b , and c , where b is the joint vertex, the vectors $\mathbf{ba} = a - b$ and $\mathbf{bc} = c - b$ are formed, and the angle θ at b is obtained as the arccosine of their normalised dot product:

$$\theta = \arccos \left((\mathbf{ba} \cdot \mathbf{bc}) / (\|\mathbf{ba}\| \cdot \|\mathbf{bc}\| + \varepsilon) \right) \quad \dots \text{ (Eq. 3.1)}$$

where $\varepsilon = 1 \times 10^{-6}$ is a small constant added to the denominator to prevent numerical division-by-zero. The resulting angle is expressed in degrees. The shoulder tilt, hip tilt, and bat swing angles were computed using the two-argument arctangent function ($\arctan2$), which returns the signed angle of a 2D vector relative to the positive x-axis, thereby preserving directional information that would be lost with a standard arctangent formulation. Stance width was computed as the absolute normalised horizontal distance between the two ankle landmarks (27 and 28).

4.5 Hybrid Feature Vector Assembly (107 Features per Frame)

The 99 raw coordinate values and 8 biomechanical angle values computed for each frame were concatenated in that order to form a single 107-dimensional hybrid feature vector per frame. This hybrid representation was designed deliberately to serve the complementary modelling requirements of the Stacking Ensemble architecture:

- The raw coordinate features preserve the full spatial layout of the skeleton, enabling the Bi-Directional LSTM to learn how landmark trajectories evolve across the 50-frame sequence — i.e., the temporal 'flow' of the shot.
- The biomechanical angle features encode clinically meaningful geometric relationships that are invariant to minor positional shifts, enabling the Random Forest and XGBoost base learners to identify discriminative postural patterns associated with each shot class.

This design decision to combine raw coordinates with derived angles — rather than relying on either representation in isolation — was motivated by prior work in pose-based action recognition demonstrating that hybrid representations consistently outperform either raw joint coordinate approaches or angle-only approaches on sports-specific classification tasks.

4.6 Temporal Resampling to Fixed Sequence Length

Cricket batting videos naturally vary in duration depending on the speed of the delivery, the type of shot played, and the recording frame rate. Raw sequences extracted from different videos therefore contain variable numbers of valid pose frames. In order to produce fixed-length input tensors required by all models in the Stacking Ensemble, a temporal resampling operation was applied to every extracted frame sequence.

Each per-video sequence of shape $(N, 107)$, where N is the number of valid frames obtained after YOLO and MediaPipe filtering, was resampled to a fixed length of 50 frames using linear interpolation independently along each of the 107 feature dimensions. Formally, given the original frame indices as a vector x_{old} of length N uniformly spaced over $[0, N-1]$, a new index vector x_{new} of exactly 50 points uniformly spaced over the same interval was constructed, and each feature channel was interpolated from x_{old} to x_{new} using NumPy's `np.interp` function. The resulting fixed-length sequence has shape $(50, 107)$.

A sequence length of 50 frames was selected as it was found empirically to provide sufficient temporal resolution to distinguish between shots of similar geometry (e.g., Cover Drive versus Cut Shot) while remaining computationally tractable for BiLSTM training. Videos yielding fewer than 10 valid pose frames after filtering — indicating severely degraded footage — were discarded entirely and did not contribute to any split.

4.7 Final Feature Tensor and Output Format

Upon completion of the feature extraction pipeline, each video in the dataset was represented as a NumPy array of shape (50, 107), where the first axis corresponds to the 50 uniformly resampled temporal frames and the second axis corresponds to the 107-dimensional hybrid feature vector. These arrays were stacked along the sample axis and saved as split-specific binary .npz files:

- `X_train.npz` — shape (2750, 50, 107) — Training samples and their feature tensors.
- `X_val.npz` — shape (590, 50, 107) — Validation samples for Meta-Learner training.
- `X_test.npz` — shape (595, 50, 107) — Held-out test samples for final evaluation.
- `y_train.npz`, `y_val.npz`, `y_test.npz` — Integer class labels (0–4) corresponding to the five sorted shot classes.
- `classes.pkl` — Serialised Python list preserving the canonical sorted class ordering: `['cover_drive', 'cut_shot', 'defense', 'pull_shot', 'sweep_shot']`.

For the tree-based base learners (Random Forest and XGBoost), the 3D tensor of shape (samples, 50, 107) was flattened along the last two axes to produce a 2D matrix of shape (samples, 5350), which was used as input during model training. The BiLSTM base learner consumed the original 3D representation directly, treating the first axis as the time-step dimension. This dual-format compatibility was a deliberate design requirement of the Stacking Ensemble architecture described in the Stacking Ensemble section.

Table 3: Feature Engineering Summary

Component	Value / Configuration	Detail
Batsman Isolation Model	YOLOv8n	Person class only, $\text{conf} \geq 0.4$, largest bbox selected
Pose Estimator	MediaPipe BlazePose	Model complexity 1, 33 landmarks, (x, y, z) per landmark
Raw Coordinate Features	99 per frame	$33 \text{ landmarks} \times 3 \text{ coordinates (x, y, z)}$
Biomechanical Angle Features	8 per frame	$4 \text{ joint angles (arccos)} + 3 \text{ directional angles (arctan2)} + 1 \text{ stance width}$
Total Features per Frame	107	$99 \text{ coordinates} + 8 \text{ biomechanical angles}$
Sequence Length (Frames)	50	Fixed via linear interpolation (temporal resampling)

Component	Value / Configuration	Detail
Minimum Valid Frames (Discard Threshold)	10	Videos below this threshold are rejected
Final Tensor Shape per Sample	(50, 107)	Frames $\times$ Features
Flattened Shape (Tree Models)	(5,350,)	50 $\times$ 107, used for RF and XGBoost
Total Dataset Samples	3,935	After cleaning, augmentation, and balancing